DEEP LEARNING · FINAL PROJECT · 2026

Reading the Khmer province on Cambodian licence plates

Five deep-learning approaches compared

26 CLASSES | 5 APPROACHES | 567 TEST CROPS



So Sereysokbotra

Lecturer: Mr. Soklong HIM · Kirirom Institute of Technology

The real-world problem: why province matters

01 · SMART ACCESS

Automated Entry

Boreys, parking & highway tolls in Cambodia need fast, hands-free vehicle verification.

02 · FATAL FLAW

Duplicate Numbers

`2A-0243` repeats across 26 provinces. Number alone causes false gate access!

03 · OCR LIMITATION

Commercial OCR Fails

Standard engines fail on Khmer script: complex stacked ligatures break character OCR.

04 · DEEP LEARNING

Whole-Line Classifier

Treat the top line as an image: **128×128 crop → 1 of 26 classes** with deep networks.

1. YOLOv10 finds the province line



2. 128 x 128 crop of the Khmer line



3. ResNet18 classifier -> 1 of 26

Takeo

class 23 of 26 p = 1.0000

End-to-end ALPR system design

01 · INPUT

Camera Stream

1080p Video Stream

LIVE FEED
30 FPS Video
Road & Barrier
Gate Monitoring

Raw Vehicle Frame

02 · DETECTION

YOLOv10 Detector

Anchor-free Plate BBox

LOCALIZATION
Latency: ~15 ms
143/149 Recall
Tight Crop Box

Cropped Plate BBox

★ THIS STUDY · 26 CLASSES

Khmer Province Classifier

ViT-B/16 & ResNet18 (128x128)

Province: "Takeo" (96.1 %)

03B · LATIN & DIGITS

CRNN + CTC Number Reader

CNN + BiLSTM + CTC Decoder

Number: "1L-4082" (83.9 %)

04 · FUSION

Plate Fusion

Canonical Formatter

STRING FUSION
Combine Lines
Syntax Check
Latency: < 1 ms

Takeo 1L-4082

05 · ACCESS

Security Gate

SQLite Whitelist

✓ ALLOW
Gate Opens

✕ DENY
Remain Closed

0 False Accepts

• **End-to-End Latency**: ~50 ms (RTX 3050) · **Parallel Recognition**: < 35 ms · **Fail-Safe Security**: 0 false accepts

Plate_v4 — Roboflow, CC BY 4.0

2 515	529	567	26	13.4 %
train crops	validation	test	classes	test near-duplicates

1 **Public benchmark dataset**
Roboflow Plate_v4 (CC BY 4.0): 3 611 vehicle crops across real Cambodian road conditions.

2 **13.4 % near-duplicate leakage**
76 of 567 test crops share source photos with training. All models also evaluated on clean n=491.

3 **Standardized crop inputs**
128×128 (CNN) / 224×224 (ViT) patches focusing exclusively on the top Khmer province line.

4 **Severe class imbalance (25:1)**
Other (453) vs. *Mondul Kiri* (18). 4 rare provinces have ≤ 50 crops, requiring **macro-F1**.

Next: **Full 26-province distribution** and minority class breakdown →

Class distribution — 26 provinces

PROVINCE	TRAIN	TEST	PROVINCE	TRAIN	TEST
Other	453	98	Kampong Chhnang	70	16
Phnom Penh	254	55	Kampot	70	15
Preah Sihanouk	148	33	Pursat	67	16
Siem Reap	147	32	Preah Vihear	65	15
Kampong Cham	110	25	Pailin	64	15
Prey Veng	101	23	Kratie	63	15
Takeo	100	23	Kampong Thom	62	14
Kandal	97	22	Tboung Khmum	61	14
Battambang	87	20	Stung Treng	52	12
Svay Rieng	86	20	**Koh Kong**	**47**	11
Banteay Meanchey	82	19	**Kep**	**42**	9
Kampong Speu	77	18	**Ratanakiri**	**20**	5
Oudor Meanchey	72	17	**Mondul Kiri**	**18**	5

• **Red bold text**: 4 minority classes with ≤ 50 training crops · *Other* (453) vs *Mondul Kiri* (18) = **25:1 imbalance** · Evaluated by **macro-F1**

Identical setup for every approach

TRAINING

1. 128×128 (CNN) / 224×224 (ViT)
2. Crop / affine / colour-jitter augmentation
3. 40 epochs · batch 32 · Adam · StepLR

EVALUATION

1. Model selected on **validation** only
2. Test set touched **once** per run
3. Accuracy + **macro-F1** (imbalance)

Seed 42

Python · NumPy · PyTorch · cuDNN · DataLoader

Colab T4

≈ 6–8 minutes per run

Only the approach changes — **that is what makes the comparison fair.**

The five approaches

	ARCHITECTURE	INITIAL WEIGHTS	TRAINING STRATEGY	PARAMETERS
A	ResNet18	Random (scratch)	All layers trainable	11.2 M
B	ResNet18	ImageNet pre-trained	Final FC only (backbone frozen)	13 k (0.12 %)
C	ResNet18	ImageNet pre-trained	Full fine-tuning (low learning rate)	11.2 M
D	Small 4-block CNN	Random (scratch)	All layers trainable	0.4 M
E	ViT-B/16 (Transformer)	ImageNet pre-trained	Full fine-tuning (196 patches)	85.8 M

01 · PRE-TRAINING

A (scratch) vs **C** (fine-tune)

Does pre-training on natural images help learn low-resource Khmer strokes?

02 · FEATURE TRANSFER

B (frozen) vs **C** (fine-tune)

Can off-the-shelf ImageNet representations read script without adaptation?

03 · MODEL CAPACITY

D (0.4M CNN) vs **A** (11.2M)

Do we actually need 11M parameters, or does a compact 4-block CNN suffice?

04 · ATTENTION VS CONV

E (ViT-B/16) vs **C** (ResNet18)

Does global patch self-attention capture script ligatures better than local convs?

Scored on the same 567 test crops

	APPROACH	VAL ACC	TEST ACC	CLEAN (N=491)	MACRO-F1	KEY OUTCOME
A	ResNet18 scratch	93.8 %	93.3 %	94.7 %	92.9 %	Strong baseline (2 515 imgs)
B	ResNet18 frozen	40.1 %	42.0 %	43.2 %	37.9 %	Domain shift collapse
C	ResNet18 fine-tune	92.6 %	94.0 %	95.3 %	92.4 %	Fast 5-epoch convergence
D	Small 4-block CNN	48.4 %	46.6 %	47.7 %	40.7 %	Under-capacity (0.4M)
E	ViT-B/16 (Transformer)	96.2 %	96.1 %	96.5 %	95.8 %	Top accuracy + macro-F1

FINDING 1 · PRE-TRAINING

A ≈ C · p = 0.63

McNemar paired test shows no statistical difference. Pre-training speeds up learning, but 2 515 crops suffice from scratch.

FINDING 2 · FEATURE TRANSFER

B Collapses to 42.0 %

Off-the-shelf ImageNet filters encode natural textures, not Khmer strokes. The network must be retrained end-to-end.

FINDING 3 · MODEL CAPACITY

D Fails at 46.6 %

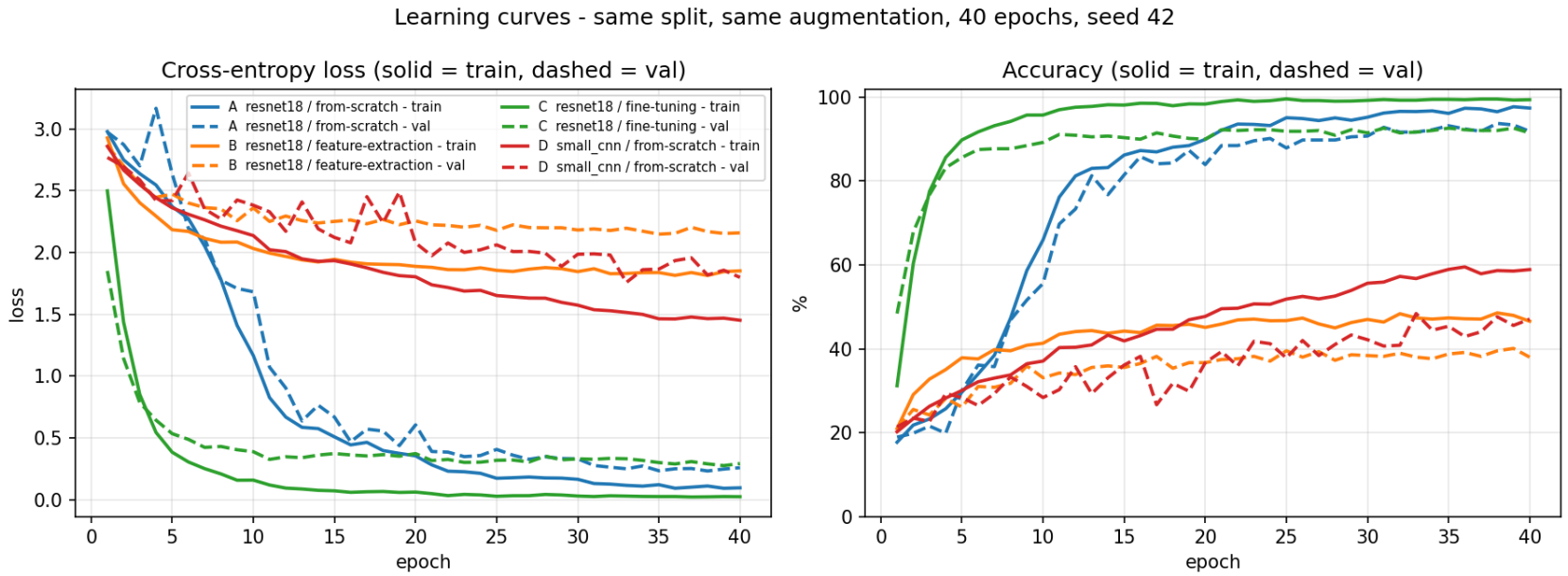
A 400k-parameter CNN severely under-fits. Separating 26 intricate glyphs requires deep residual or attention capacity.

FINDING 4 · TRANSFORMERS

ViT Wins: 96.1 % / 95.8 %

Self-attention captures full character ligatures and improves inverted plate reading from 19.4 % to **26.8 %**.

Learning curves



5 vs 20
epochs to 85 % — C vs A

97–99 %
train acc A, C — mild over-fit

< 60 %
train acc B, D — under-fit

Tuning C — selected on validation

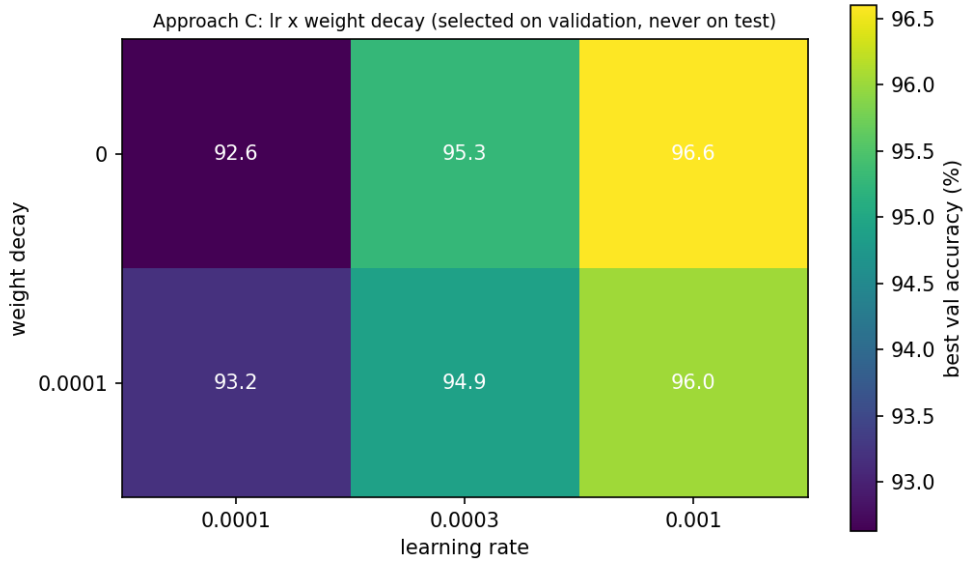
LEARNING RATE	WD = 0	WD = 1E-4
1e-4	92.6 %	93.2 %
3e-4	95.3 %	94.9 %
1e-3	**96.6 % ← selected**	96.0 %

1. Textbook "low LR for fine-tuning" was the **worst** cell
2. 10× larger LR → **+4 points** validation accuracy
3. Weight decay: no help — augmentation already regularises

96.5 %

final model — test accuracy · lr 1e-3, wd 0

VALIDATION ACCURACY · LR × WEIGHT DECAY



Where the 34 errors come from

8 of 34

one pair — Banteay ↔ Oudor Meanchey

1. បៈមានជ័យ vs ឱៈមានជ័យ — **one glyph** apart
2. Other errors also share a whole word
3. Rest: blur, 20-px text, JPEG artefacts
4. Smallest classes **90 %** vs 94.2 %

MOST CONFIDENT WRONG PREDICTIONS · C



Why — in course concepts

B → 42 %	DOMAIN SHIFT	ImageNet filters are not Khmer strokes
A ≈ C	DATA SIZE	2 515 images suffice from random init
D → 47 %	CAPACITY	0.4 M params cannot separate 26 words
E → 96.1 %	ATTENTION	global self-attention captures full glyph ligatures
tuning +2.5	OPTIMISATION	fine-tuning wants a training-size LR

What this study cannot claim

1. Public scraped data — not purpose-collected
2. **13.4 %** near-duplicates in the test split
3. 567 crops → **± 2 points**; A vs C not separable
4. One seed per configuration
5. CNNs and Vision Transformers evaluated (2 architecture families)

Four takeaways

01

Train the whole network — frozen features do not transfer (42 %)

02

Transformers excel on script — ViT matches 96.1 % & lifts upside-down to 26.8 %

03

Capacity matters — 0.4 M parameters is not enough; 11 M to 86 M is

04

For CNNs, biggest gain came from **tuning the learning rate**

Next: re-split by source photo · photograph the five smallest provinces · benchmark Swin Transformer & edge quantization

QUESTIONS WELCOME

Thank you

Every number came from a script in the repository.

github.com/SoSereysokbotra



APPENDIX

Supporting material



full results · statistics · tuning grid · the gate system · confusion · per-class · measurement · code

Full results — every column

	APPROACH	VAL	TEST	CLEAN N=491	MACRO-F1	TRAINABLE	TIME
A	ResNet18 scratch	93.8 %	**93.3 %**	94.7 %	92.9 %	11 189 850	6.1 min
B	ResNet18 frozen	40.1 %	**42.0 %**	43.2 %	37.9 %	13 338	5.3 min
C	ResNet18 fine-tune	92.6 %	**94.0 %**	95.3 %	92.4 %	11 189 850	5.9 min
D	small CNN	48.4 %	**46.6 %**	47.7 %	40.7 %	396 058	5.3 min
E	ViT-B/16 fine-tune	96.2 %	**96.1 %**	96.5 %	95.8 %	85 818 650	8.0 min

1. Model chosen on **val**; test read **once** per run
2. Clean subset = the 491 crops with no shared source photo — accuracy is *higher*, so the leak did not inflate anything
3. All runs: 40 epochs, batch 32, seed 42, Colab T4

Is the A–C difference real?

McNemar exact test, paired on the same 567 crops

	C RIGHT	C WRONG
A right	512	17
A wrong	21	17

1. 38 discordant pairs → **p = 0.627**
2. **Not significant** — A and C are a tie
3. I do not claim a winner between them

95 % Wilson confidence intervals (n = 567)

APPROACH	TEST	95 % CI	WIDTH
A	93.3 %	90.9 – 95.1	4.1 pts
B	42.0 %	38.0 – 46.1	8.1 pts
C	94.0 %	91.7 – 95.7	3.9 pts
D	46.6 %	42.5 – 50.7	8.2 pts
E	96.1 %	94.2 – 97.4	3.2 pts

A, C and E overlap in high confidence — which is why **567 images cannot separate top models.**

Full tuning grid — all six runs

LR	WEIGHT DECAY	BEST EPOCH	VALIDATION	TEST	MACRO-F1
1e-3	**0**	33	**96.60 % ← selected**	96.47 %	96.04 %
1e-3	1e-4	35	96.03 %	95.41 %	95.06 %
3e-4	0	15	95.27 %	**97.00 %**	96.55 %
3e-4	1e-4	27	94.90 %	95.94 %	95.35 %
1e-4	1e-4	36	93.19 %	94.53 %	93.02 %
1e-4	0	31	92.63 %	94.00 %	92.44 %

! **Yes — 3e-4 scores higher on test (97.0) than the model I selected.** Selecting it *because* it won on test would be selecting on the test set — the exact mistake this project is about avoiding. Validation chooses; test is reported once.

The ALPR system this model serves

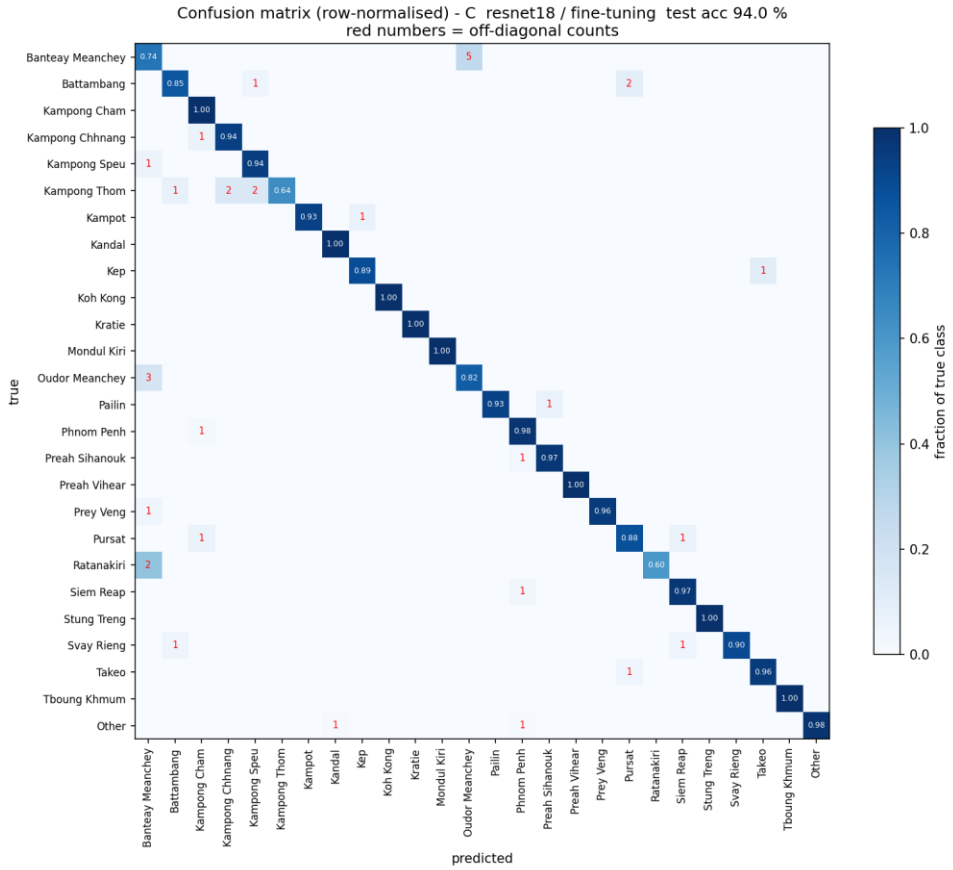


END-TO-END ON REAL SCENES

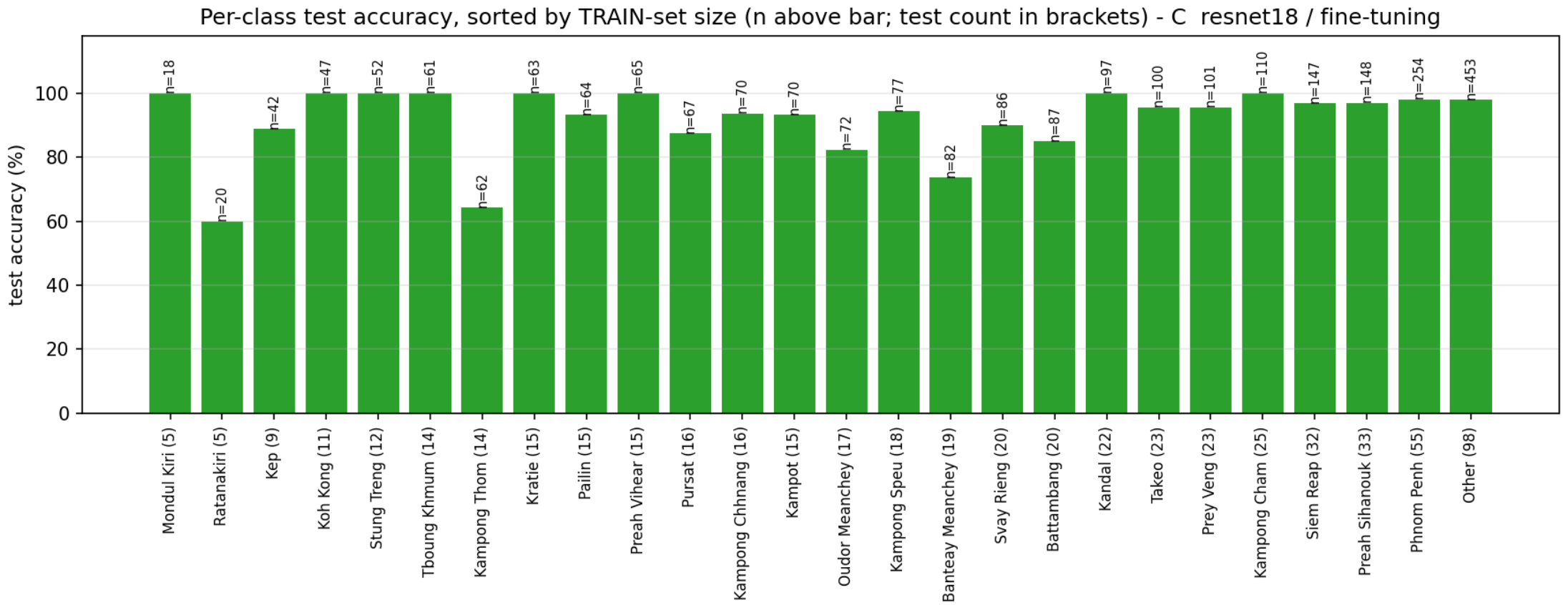
143/149	83.9 %	94.0 %	0	~50 ms
plate detected	number correct	province correct	false accepts	per frame

149 labelled real scenes · RTX 3050 · whitelist of 137 near-miss plates

Confusion matrix — approach C



Per-class accuracy vs training-set size



Measurement discipline

01

CRNN test set: **34.9 %** plate-level leakage found

02

Headline fell 69.1 → **51.7 %** — the lower number is reported

03

Spatial Transformer proved to do **nothing** — negative result

04

`experiment_log.csv` — every measurement tagged with its commit

Laroca et al. (IJCNN 2023) report 19–68 % duplicates in standard LPR benchmarks.

The code behind B and D

Feature extraction (B) — only the head trains

```
for name, prm in model.named_parameters():
    if not name.startswith('fc.'):
        prm.requires_grad = False      # 11 176 512 frozen
    else:
        trainable += prm.numel()       # 13 338 trainable

optimizer = Adam(
    [p for p in model.parameters() if p.requires_grad],
    lr=args.lr, weight_decay=args.weight_decay)
```

SmallCNN (D) — 4 blocks, 128→64→32→16→8, then GAP

```
def block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1),
        nn.BatchNorm2d(cout), nn.ReLU(True),
        nn.MaxPool2d(2))

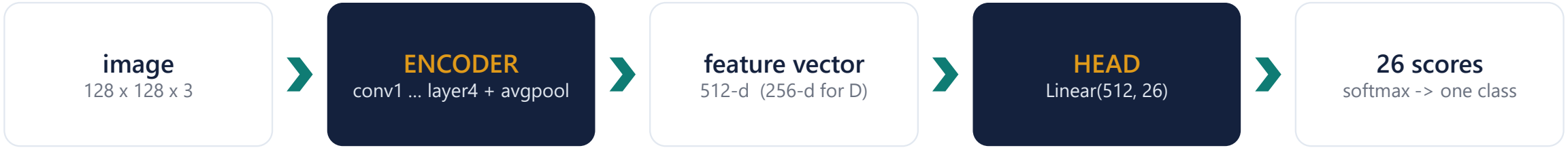
self.features = nn.Sequential(
    block(3,32), block(32,64), block(64,128), block(128,256))
self.pool = nn.AdaptiveAvgPool2d(1)   # 396 058 params total
```

Seed 42 in Python, NumPy, PyTorch, cuDNN and the DataLoader

`last.pth` saved every epoch + `--resume` for Colab disconnects

The lr 1e-4 grid cell reproduced run C **to the digit**

Encoder, head — and why there is no decoder



APPROACH	ENCODER	FEATURE	HEAD	DECODER
A / B / C	ResNet18 conv stack	512-d	Linear(512, 26)	none
D	4 conv blocks + GAP	256-d	Linear(256, 26)	none
E	PatchEmbed + 12 Transformer blocks	768-d (CLS)	Linear(768, 26)	none

1 A **decoder generates a sequence**, one step at a time, each step conditioned on the previous one

2 My output is **one label out of 26** — there is nothing to generate, so a decoder would be architecturally meaningless

! `CTCDecoder` in the number reader is **not a neural decoder** — it is a collapse rule: argmax per timestep, merge repeats, drop blanks. **No weights, never trained.**